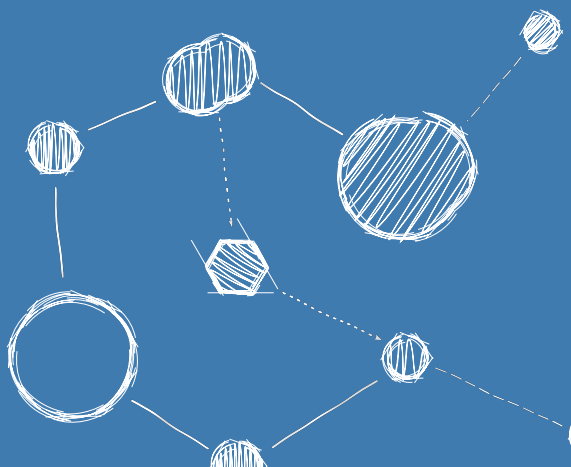
institut für astrophysik
UNIVERSITÄTSSTERNWARTE WIEN

universität wien

FLIGHTOS – AN RTOS FOR SPACE-BASED ASTRONOMY

Performance Figures



Performance Figures

Reference: FLIGHTOS-UVIE-PERF-001

Version: Issue 0.2, August 31, 2026

Prepared by: Armin Luntzer[1]

Checked by: Roland Ottensamer[1]

Approved by: Franz Kerschbaum[1]

Authorised by: -

[1] Department of Astrophysics, University of Vienna

Contents

Revision History

Revision	Date	Author(s)	Description
0.1	19.01.2021	AL	Initial version
0.2	31.08.2026	AL	update document build

1. Introduction

1.1 Purpose of the Document

This document is a collection of various performance figures of the FlightOS operating system on target platforms such as the GR712 or GR740.

2. Applicable and Reference Documents

[1] *GR712RC Dual-Core LEON3FT SPARC V8 Processor User's Manual*. 2016.

3. Terms, Definitions and Abbreviated Items

4. Timing System

4.1 Overview

The internal timekeeping system in FlightOS relies upon a hardware clock device which returns the current system uptime in seconds and nanoseconds. This clock device is implemented in the architecture specific section with an interface compatible to *struct clocksource* and registered to the main kernel *ktime* interface by the architecture specific bootup sequence.

Upon registration of the clocksource, the ktime subsystem calibrates the uptime clock readout overhead by reading the uptime clock for a configurable number of repetitions. For each readout, the Δt to the previous update is calculated and accumulated. The accumulated Δt is the averaged over the number of loops and divided by 2 to account for the period between two successive readouts. This value is then stored and added to all timesamps retrieved by calls to *ktime_get()* as a correcting offset.

Another timing-related kernel subsystem are *clockevent* devices, which are registered by the architecture specific bootup code. These devices can have properties such as periodic and one-shot timeouts, or absolute timings given a particular ktime.

Available clockevent devices are offered to the *tick* subsystem by the architecture code for each booting SMP CPU to support thread scheduling. The tick system checks the device properties and may also exchange an already registered clockevent device for one with better characteristics. When a clockevent device is registered, its minimum processable tick length is calibrated and also serves as a benchmark of interrupt jitter.

4.2 Performance Figures: GR712

In the GR712, the *General Purpose Timer Unit with Time Latch Capability*[1] is used as the architecture-specific uptime clock. The prescaler for this timer is set to 4 by default. One of the timers is configured to underflow once per second. The needed value depends on the CPU clock frequency and therefore the calculated baseline precision of the clock is *(4 + 1) / freq)* or 500 ns for a system clock of 10 MHz.

The measured precision is however lower, as it is increased by the instruction in the code path to read out the uptime clock.

For the *GR712RC Dual-Core LEON3-FT Development Board* at a clock speed of 80 MHz, the total readout overhead amounts to < 850 ns.

For the *GR712 CHEOPS DPU prototype board* clocked at 25 MHz the overhead is < 2700 ns.

5. Interrupt Timings

5.1 Overview

Interrupts and interrupt handling timings as well as their jitter depend on a multitude of factors such as concurrent number of raised interrupts, memory bus loads, states of instruction and data caches and so on. This chapter is only concerned with the software related effects.

The *tick* subsystem of the kernel can be used to establish a baseline of interrupt jitter and predictability, as the clockevent devices (4) are calibrated at bootup. This is done by registering a special tick calibration hander to the ISR-driven callback of the clockevent device. The clockevent timeout is then programmed to the calibrated kernel uptime clock readout overhead and the minimum tick period of the device is monitored for each pass of a loop, which increased the clockevent timeout by the uptime clock overhead until the minimum tick period parameter for the device is set by the calibration callback handler. This is done because a clockevent device is required to cause a timer IRQ to be raised every time a timeout is programmed, regardless of the underlying precision of the clock. That means that even a timeout of 0 must result in an (immediate) interrupt).

When a value for the minimum tick period is established, a calibration loop is executed to establish an average minimum period by repeating the above process multiple time for the found timeout period. The average minimum tick timeout is then doubled for extra margin and set in the configuration paramters of the clockevent device.

The values for the minimum tick period represent the lower limit of the achievable scheduling event cadence for the given system configuration.

5.2 Performance Figures: GR712

On the GR712, the *General Purpose Timer Unit*[1] is used for the architecture-specific clockevent devices. The prescaler for these timers is set to 7 by default. The timer precision also depends on the CPU clock frequency and therefore the calculated baseline precision of these clockevent devices is *(7 + 1) / freq)* or 800 ns for a system clock of 10 MHz.

The measured minimum timeout is however much higher, as it is defined by the instructions executed in the code path of the clockdevice implmentation and the setup of the interrupt context in the CPU.

For the *GR712RC Dual-Core LEON3-FT Development Board* at a clock speed of 80 MHz, the timer units calibrate to 10 ± 0.2 µs and 12 ± 0.5 µs for CPUs 0 and 1 respectively.

The same test was run on the *GR712 CHEOPS DPU prototype board* clocked at 25 MHz and resulted in 30 ± 0.3 µs and 32 ± 0.3 µs.

Note that the above values are doubled in the software for extra safety margin.

6. Scheduling

6.1 Overview

TBW

6.2 Performance Figures: GR712

GR712RC Dual-Core LEON3-FT Development Board clocked at 80 MHz, test setup: To determine the absolute scheduling precision an *Earliest Deadline First* real-time thread was configured for a 1 second period, with a allocated WCET of 2 ms and a deadline of 3 ms. In parallel and on the same CPU, a high frequency EDF thread for flushing the tty character queue to the UART for printout of result was configured at a period of 500 µs, WCET 50µs and deadline of 450ms to create at least scheduling events (and therefore timer interrupt requests) at a minimum rate of 2 kHz. (Note: the typical values for the tty queue are P=10 ms, WCET=50 µs and D=9 ms)

A long duration test (> 1hr) yielded a maximum peak-to-peak jitter of <= 2 µs with an average <= 1 µs for the absolute scheduling period since the start of the thread and is therefore on the scale of the readout overhead and jitter of the main system uptime clock. This value also represents the scale of interrupt jitter in general.

For the *GR712 CHEOPS DPU prototype board* at a clock speed of 25 MHz, the tty queue RT thread parameters were increased by a factor of 4 to account for the inherently lower timing resolution due to the system clock speed. The configuration of the thread measuring the timing precision was kept the same as with the 80 MHz clock. A maximum peak-to-peak jitter of <= 4µs with an average of <= 1µs was observed during a long duration test.